

浙江工业大学

计算机组成原理课程设计

报告题目：TD-CMA 复杂模型机中断 GCD 程序设计



学 号 XXXXXXXXXXXXXXXX

姓 名 XXX

班 级 XX 学院 XX 专业 XX 班

提交日期 20XX 年 X 月

目 录

一、 实验目的	1
二、 实验内容	1
三、 实验原理	3
3.1 指令系统与分析	3
3.2 指令框图及分析	12
3.3 指令系统对应微程序二进制代码实现及分析	14
3.4 机器程序及分析	17
四、 实验步骤	20
4.1 微程序写入及校验	20
4.2 机器程序写入及校验	21
4.3 程序运行及调试	21
五、 实验结果及分析	21
5.1 演示程序一	22
5.2 演示程序二	22
5.3 结果分析	23
六、 实验问题及思考	23
七、 实验总结	24
7.1 验收答辩问题回答	24
7.2 实验心得体会	26

TD-CMA 复杂模型机中断 GCD 程序设计

一、实验目的

综合运用所学计算机组成原理知识，在 TD-CMA 复杂模型机平台上设计并实现较为完整的计算机系统程序。本次课程设计以“中断 GCD 程序”为主题，通过修改和使用模型机指令系统、编写机器程序和分析微程序执行过程，实现带中断处理能力的更相减损法求最大公约数功能。

本实验的目的主要包括以下几个方面：

1. 理解并掌握计算机的整机结构，熟悉 TD-CMA 复杂模型机中运算器、控制器、寄存器组、主存、I/O 接口和系统总线之间的连接关系与工作过程。
2. 掌握基本功能模块的工作原理和操作方法，能够理解 PC、IR、AR、SP、通用寄存器、暂存器 A/B、ALU、标志位 FC/FZ 等部件在指令执行过程中的作用。
3. 掌握控制器的微程序实现方法，理解机器指令和微指令之间的关系，能够根据一条机器指令分析其对应的微操作流程。
4. 掌握复杂模型机指令系统的基本格式，能够分析 LDI、IN、OUT、MOV、SUB、JMP、BZC、STI、CLI、IRET 等指令的机器码含义。
5. 理解中断系统的基本工作过程，掌握 8259 中断控制器初始化、中断向量设置、开中断、关中断、中断响应和中断返回等内容。
6. 理解堆栈在中断处理中的作用，明确 SP 初始化、中断返回地址保存以及 IRET 恢复 PC 的基本过程。
7. 训练逻辑分析和故障排查能力，能够通过单步运行观察寄存器、PC、标志位和端口输出状态，定位机器程序或微程序中的错误。

二、实验内容

本课程设计围绕 TD-CMA 复杂模型机展开，主要完成以下三项内容：

1. 设计并实现一套完整的指令系统；
2. 设计并实现完整的计算机，即采用上述指令系统的复杂模型机；
3. 利用该计算机实现带中断系统的更相减损法求最大公约数。

围绕上述三项内容，本实验进一步展开如下具体设计与实现工作：

- 设计并实现一套能够完成本项目功能的指令系统。根据 GCD 运算和中断处理的需要，分析并使用 LDI、IN、OUT、MOV、SUB、JMP、BZC、STI、CLI、IRET 等指令。其中，本项目将原指令系统中 0000 入口对应的运算功能改造为减法，使其能够支持更相减损法中的核心相减操作。
- 设计并实现完整的模型机程序。主程序从 00H 单元开始，负责初始化 8259 中断控

制器、设置中断向量相关信息、设置屏蔽字、初始化堆栈指针 SP、执行 STI 开中断，并通过 JMP 13H 进入等待循环。

- 设置中断向量。在 30H 单元写入 40H，即 \$P 30 40，表示 IR0 中断发生后，CPU 根据中断向量进入 40H 单元执行中断服务程序。这样主程序不需要直接跳转到服务程序，而是由外部中断触发进入服务程序，更能体现中断机制。
- 编写中断服务程序。中断服务程序从 40H 单元开始，首先执行 CLI 关中断，避免服务程序执行过程中再次响应中断；然后通过 IN R1,00H 和 IN R2,00H 从输入端口读入两个数，分别存入 R1 和 R2。
- 利用复杂模型机实现带中断系统的更相减损法求最大公约数。程序使用 R3 作为比较临时寄存器，通过 SUB R3,R1 产生 FC/FZ 标志位，并利用 BZC 判断是否需要跳转；使用 R0 作为交换临时寄存器，实现 R1 和 R2 的交换；通过循环相减直到两个数相等。
- 完成结果输出和中断返回。当计算结束后，最大公约数保存在 R1 中，程序执行 OUT 40H,R1 将结果输出到端口 40H，随后执行 IRET，从堆栈恢复返回地址，使 CPU 回到主程序继续等待下一次中断。
- 对机器程序和微程序进行写入、校验和调试。重点检查主程序地址、中断向量地址、服务程序入口地址、BZC 跳转地址、SUB 微程序减法功能、STI/CLI 中断控制以及 IRET 返回过程是否正确。

通过以上内容，本课设最终实现了从主程序初始化、中断触发、输入数据、GCD 运算、结果输出到中断返回的完整流程，能够较好地体现复杂模型机指令系统、微程序控制、中断响应和堆栈返回机制之间的关系。形象地，具体实验内容如下所示

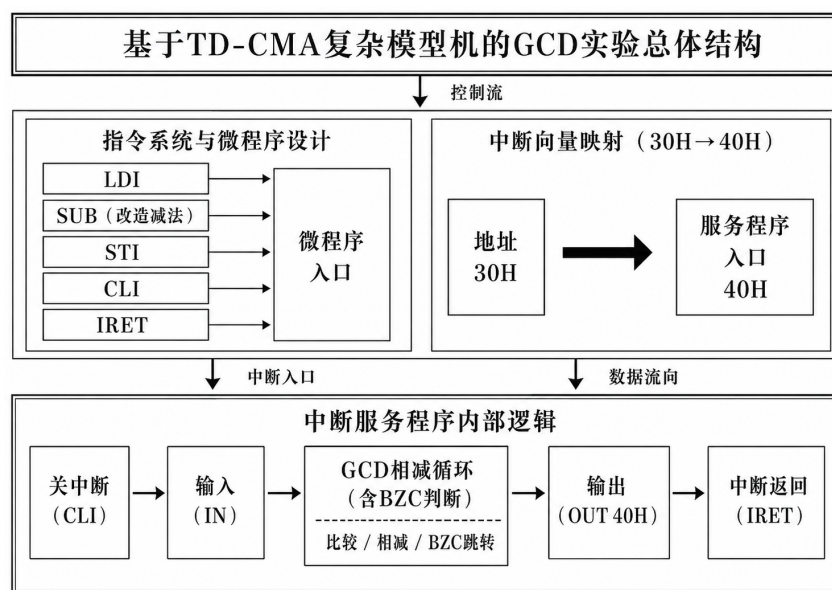


图 1 实验内容图

三、实验原理

3.1 指令系统分析

3.1.1 指令格式

TD-CMA 复杂模型机的机器指令通常由单字节指令、双字节指令以及微指令配合完成。机器指令用于描述程序员希望 CPU 完成的操作，微指令用于在控制器内部产生更具体的控制信号。本节主要对本课设中用到的指令格式进行说明。

(1) 单字节指令格式

单字节指令（ADD、AND、INC、SUB、OR、RR、HLT、MOV）一般只需要一个字节即可说明操作类型和寄存器编号。其基本格式如表 1 所示。

表 1 单字节指令格式

I7	I6	I5	I4	I3	I2	I1	I0
OP-CODE				RS		RD	

其中，OP-CODE 表示操作码，用来确定当前指令执行什么操作；RS 表示源寄存器编号，RD 表示目的寄存器编号。比如 MOV RD,RS 的含义就是将 RS 所指定寄存器的内容送入 RD 所指定的寄存器。

(2) RS 或 RD 选定的寄存器

RS 和 RD 字段均为 2 位，因此最多可以表示 4 个通用寄存器。寄存器编码如表 2 所示。

表 2 RS 或 RD 选定的寄存器

RS 或 RD	00	01	10	11
寄存器	R0	R1	R2	R3

例如，机器码 4BH 的二进制形式为 0100 1011，其中高四位 0100 表示 MOV，中间两位 10 表示源寄存器 R2，低两位 11 表示目的寄存器 R3，因此该指令为：

MOV R3, R2

其功能是：

$R2 \rightarrow R3$

(3) IN 和 OUT 的指令格式

IN 和 OUT 指令属于双字节指令。第一字节说明操作码和寄存器信息，第二字节给出端口地址 P。其格式如表 3 所示。

表 3 IN 和 OUT 的指令格式

第一字节								第二字节
I7	I6	I5	I4	I3	I2	I1	I0	I7~I0
OP-CODE				RS		RD		P

本设计中，IN R1,00H 的机器码为：

\$P 41 21, \$P 42 00

其含义是从端口 00H 输入数据到 R1。同理，OUT 40H,R1 的机器码为：

\$P 4E 34, \$P 4F 40

其含义是将 R1 中的 GCD 结果输出到端口 40H。

(4) LDI 指令格式

除了普通的单字节寄存器指令外，本课设中还用到了 LDI 这类双字节指令。双字节指令的特点是第一字节说明操作码和寄存器字段，第二字节给出具体的数据或地址。LDI 指令中的第二字节是立即数 Data，用于直接装入指定寄存器。LDI 指令格式如表 4 所示。

表 4 LDI 指令格式

第一字节								第二字节
I7	I6	I5	I4	I3	I2	I1	I0	I7~I0
OP-CODE				RS		RD		Data

LDI 指令的功能是把第二字节的立即数送入指定寄存器。例如本课设主程序开头：

\$P 00 60, \$P 01 13

合起来表示：

LDI R0,13H

其含义是把立即数 13H 装入 R0。后续程序再通过 OUT C0H,R0 将 R0 中的数据写入 8259 的控制端口，用于完成中断控制器初始化。

本课设中还使用了：

\$P 10 63, \$P 11 A0

这条指令在本中断模型机中解释为 LDI SP,A0H, 用于将堆栈指针 SP 初始化为 A0H, 为中断响应时保存返回地址做准备。

(5) LAD、STA、JMP、BZC 指令格式

LAD、STA、JMP、BZC 这类指令也属于双字节指令。它们的第一字节包含操作码 OP-CODE、寻址方式 M 和寄存器字段 RD；第二字节为地址字段 D。CPU 执行这类指令时, 会根据 M 字段和 D 字段形成有效地址 E。

LAD、STA、JMP、BZC 的指令格式如表 5 所示。

表 5 LAD、STA、JMP、BZC 指令格式

第一字节								第二字节
I7	I6	I5	I4	I3	I2	I1	I0	I7~I0
OP-CODE				M		RD		D

其中, 高四位 OP-CODE 用来区分指令类型。例如 JMP 的操作码为 1110, BZC 的操作码为 1111。中间两位 M 表示寻址方式, 决定第二字节 D 应该如何解释。低两位 RD 在 LAD、STA 中一般用于指定寄存器, 而在 JMP、BZC 中通常不起主要作用。

本课设主要使用了其中的 JMP 和 BZC 指令。JMP 用于主程序等待循环, 例如 JMP 13H; BZC 用于根据 FC/FZ 标志位进行条件转移, 例如 BZC 51H 和 BZC 4EH。

(6) 寻址方式与本课设中的转移指令分析

寻址方式 M 决定了地址字段 D 如何转换为最终使用的有效地址 E。对于 JMP 和 BZC 这类转移指令来说, 寻址方式会直接影响 PC 的装入结果。寻址模式如表 6 所示。

表 6 寻址模式表

寻址模式 M	有效地址 E	说明
00	$E=D$	直接寻址
01	$E=(D)$	间接寻址
10	$E=(RI)+D$	RI 变址寻址
11	$E=(PC)+D$	相对寻址

本课设中, JMP 13H 和 BZC 51H 都采用直接寻址, 即 M=00。直接寻址时, 有效地址就是第二字节给出的地址字段 D, 因此比较容易分析。

例如主程序等待循环中的两条机器码为:

\$P 13 E0, \$P 14 13

其中，E0H=1110 0000，高四位 1110 表示 JMP，中间两位 M=00 表示直接寻址，第二字节 13H 是目标地址。由于直接寻址时 E=D，所以该指令执行后：

$$PC \leftarrow 13H$$

也就是说，主程序会一直回到 13H 单元执行，形成等待中断的循环。

再看中断服务程序中的条件转移：

\$P 47 F0, \$P 48 51

其中，F0H=1111 0000，高四位 1111 表示 BZC，中间两位 M=00 表示直接寻址，第二字节 51H 是目标地址。因此这两条机器码合起来表示：

BZC 51H

BZC 的含义是：如果上一条 SUB 指令使 FC=1 或 FZ=1，则将有效地址 E 装入 PC，也就是跳转到 51H 单元；如果 FC 和 FZ 都不为 1，则不跳转，程序继续顺序执行。这个设计正好用于本课设中的大小比较和交换分支判断。

例如程序在 46H 执行 SUB R3,R1 后，如果发现 R2 小于或等于 R1，则 FC 或 FZ 会被置 1，接着 47H 处的 BZC 51H 就会跳转到交换分支。这样程序就能在没有专门比较指令的情况下，利用减法结果和标志位完成条件判断。

(7) 微指令格式

微指令是控制器发出的基本控制命令，描述的是机器指令所描述操作的进一步具体动作。本实验中，一条微指令为 24 位，通常用 6 个十六进制数表示。其格式如表 7 所示。

表 7 微指令格式

23	22	21	20	19
M23	INTA	WR	RD	IOM
18~15	14~12	11~9	8~6	5~0
S3~S0	A 字段	B 字段	C 字段	MA5~MA0

其中，WR、RD、IOM 用于控制主存或 I/O 读写；S3~S0 用于选择 ALU 的运算功能；A 字段控制数据写入位置；B 字段控制数据来源；C 字段控制判别条件或特殊控制；MA5~MA0 表示下一条微指令地址。为了便于后续分析，A、B、C 字段的编码分别列于表 8、表 9 和表 10。

表 8 A 字段编码表

14	13	12	选择
0	0	0	NOP
0	0	1	LDA
0	1	0	LDB
0	1	1	LDRi
1	0	0	LDSP
1	0	1	LOAD
1	1	0	LDAR
1	1	1	LDIR

A 字段主要控制“总线上的数据写到哪里”。例如 LDA 表示将总线数据送入暂存器 A，LDB 表示送入暂存器 B，LDRi 表示将数据写回相应寄存器。

表 9 B 字段编码表

11	10	9	选择
0	0	0	NOP
0	0	1	ALU_B
0	1	0	R0_B
0	1	1	PD_B
1	0	0	RI_B
1	0	1	SP_B
1	1	0	PC_B
1	1	1	保留

B 字段主要控制“谁把数据送到总线”。例如 ALU_B 表示 ALU 输出送总线，PC_B 表示 PC 内容送总线，SP_B 表示堆栈指针 SP 内容送总线。

表 10 C 字段编码表

8	7	6	选择
0	0	0	NOP
0	0	1	P<1>
0	1	0	P<2>
0	1	1	P<3>
1	0	0	P<4>
1	0	1	LDPC
1	1	0	STI
1	1	1	CLI

C 字段主要用于微程序流程控制和特殊控制。比如 P<1>、P<2>、P<3>、P<4> 用于微程序判别，LDPC 用于将目标地址装入 PC，STI 和 CLI 分别用于开中断和关中断。

本课设中，SUB 指令的微操作可以理解为：

$$RD \rightarrow A$$

$$RS \rightarrow B$$

$$A - B \rightarrow RD$$

以 SUB R2,R1 为例，就是先把 R2 送入暂存器 A，再把 R1 送入暂存器 B，最后由 ALU 执行 $A - B$ ，并把结果写回 R2。因此，SUB 能否正确执行，关键在于微指令中的 S3~S0 是否选择了减法功能。

(8) 机器连线图

复杂模型机的程序运行不仅依赖机器码和微程序，也依赖实验箱各模块之间的正确连线。机器连线图主要反映 8259 中断控制单元、时序与操作台单元、微程序控制单元、ALU 与寄存器单元、PC 与 AR 单元、主存单元和 I/O 单元之间的连接关系。

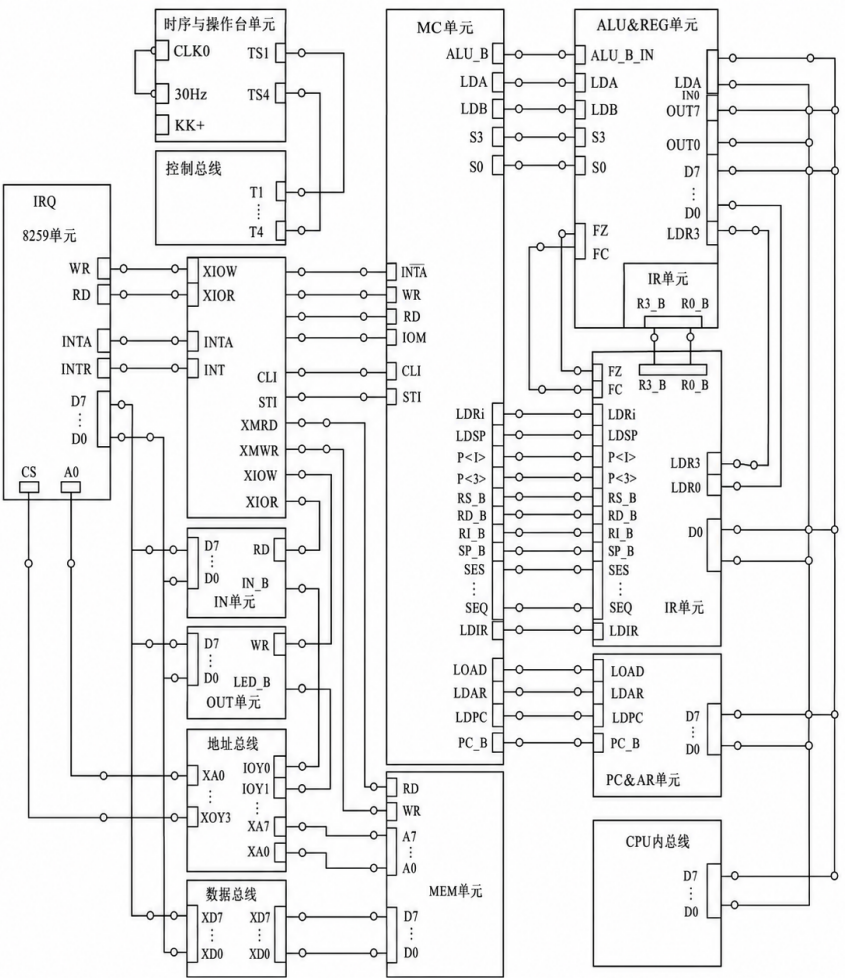


图 2 TD-CMA 带中断的复杂模型机实验连线图

从图 2 可以看出，模型机内部主要通过控制总线、地址总线和数据总线连接各个功能模块。控制总线传递 WR、RD、IOM、INTA、STI、CLI 等控制信号；地址总线用于选择主存或 I/O 端口；数据总线用于在寄存器、主存、I/O 和 ALU 之间传送数据。

本课设中，中断部分主要依赖 8259 单元与 CPU 控制部分之间的连接。按下 KK/IR0 后，8259 产生中断请求信号，CPU 在开中断状态下响应该请求，并根据中断向量进入 40H 单元的中断服务程序。GCD 运算部分则主要依赖 ALU 与寄存器单元，R1、R2、R3 和 R0 分别承担输入、比较、相减和交换等功能。

(9) 微指令判别条件

微程序并不是始终按顺序执行。对于不同机器指令，控制器需要根据操作码、寻址方式、标志位和中断状态决定下一条微指令地址。P<1>、P<2>、P<3>、P<4> 就是用于改变微程序流程的判别条件。

表 11 指令格式位说明

I7	I6	I5	I4	I3	I2	I1	I0
最高位	第 7 位	第 6 位	第 5 位	第 4 位	第 3 位	第 2 位	最低位

P<1> 主要根据机器指令操作码进行分支。微程序通过指令寄存器 IR 中的 I7~I0 判断当前指令属于哪一类，再进入对应的微程序入口。其散转下址生成规则如表 12 所示。

表 12 P<1> 散转下址生成规则

I7I6	SE5	SE4	SE3	SE2	SE1	SE0
≠11	MA5	MA4	I7	I6	I5	I4
=11	MA5	MA4	1	1	I3	I2

P<2> 主要用于寻址方式判别。对于 LAD、STA、JMP、BZC 等带 M 字段的指令，需要根据 M 字段形成有效地址 E。P<2> 的散转下址生成规则如表 13 所示。

表 13 P<2> 散转下址生成规则

SE5	SE4	SE3	SE2	SE1	SE0
MA5	MA4	MA3	MA2	I5	I4

P<3> 与条件转移有关，是本课设中比较关键的判别条件。BZC 指令要求当 FC=1 或 FZ=1 时跳转，因此 P<3> 判别需要结合标志位 FC 和 FZ。其逻辑表达式和散转下址生成规则如下：

$$SE4 = \overline{P_3} \cdot (FZ + FC)$$

表 14 P<3> 散转下址生成规则

SE5	SE4	SE3	SE2	SE1	SE0
MA5	FC 或 FZ	MA3	MA2	MA1	MA0

在本课设中，P<3> 主要用于两处 BZC 判断。第一处是 47H 的 BZC 51H，用于判断是否进入交换分支；第二处是 4AH 的 BZC 4EH，用于判断相减结果是否为 0 并跳转到输出位置。也就是说，更相减损法能否正常循环，很大程度上取决于 FC/FZ 标志位和 P<3> 判别是否正确。

P<4> 用于中断判别。当外部中断请求 INTR 到来，并且中断允许标志 EI 为 1 时，CPU 才会进入中断响应流程。其逻辑表达式和散转下址生成规则如下：

$$SE5 = \overline{P_4} \cdot INTR \cdot EI$$

表 15 P<4> 散转下址生成规则

SE5	SE4	SE3	SE2	SE1	SE0
INTR 与 EI	MA4	MA3	MA2	MA1	MA0

本课设中，主程序执行 STI 后打开中断，使 EI=1；按下 KK/IR0 后产生中断请求，P<4> 判别成立，CPU 才根据中断向量进入 40H 单元的中断服务程序。服务程序入口处执行 CLI，关闭中断，避免在 GCD 运算过程中再次响应中断。

3.1.2 指令分析

本课设用到的指令主要包括数据传送、输入输出、算术运算、程序转移和中断控制几类。表 16 对主要指令的汇编符号、指令格式和功能进行整理，其中红色部分表示在原有指令系统基础上新增或改造的指令。

表 16 本课设主要指令分析

汇编符号	指令格式					指令功能
MOV RD,RS	0100	RS	RD			$RS \rightarrow RD$
SUB RD,RS	0000	RS	RD			$RD - RS \rightarrow RD$
CLI	1000	**	**			CPU 关中断
XOR RD,RS	1001	RS	RD			$RD \oplus RS \rightarrow RD$
STI	0111	**	**			CPU 开中断
IRET	1011	**	**			中断返回
LAD M,D,RD	1100	M	RD	D		$E \rightarrow RD$
STA M,D,RS	1101	M	RD	D		$RD \rightarrow E$
JMP M,D	1110	M	**	D		$E \rightarrow PC$
BZC M,D	1111	M	**	D		当 FC=1/FZ=1 时, $E \rightarrow PC$
IN RD,P	0010	**	RD	P		$[P] \rightarrow RD$
OUT P,RS	0011	RS	**	P		$RS \rightarrow [P]$
LDI RD,D	0110	**	RD	D		$D \rightarrow RD$
HALT	0101	**	**			停机

从表 16 可以看出，本课设实际使用的核心指令为 LDI、IN、OUT、MOV、SUB、JMP、BZC、STI、CLI 和 IRET。其中，LDI 和 OUT 主要用于 8259 初始化和 SP 设置；IN 用于输

入数据；MOV 用于寄存器复制和交换；SUB 用于完成更相减损法中的相减操作；BZC 依据 FC/FZ 标志位实现条件转移；STI、CLI 和 IRET 用于中断控制和中断返回。

本课设在原有指令系统基础上进行了适当调整：将原 0000 运算入口改造为 **SUB RD,RS**，实现 $RD - RS \rightarrow RD$ ；新增 **XOR RD,RS**，实现 $RD \oplus RS \rightarrow RD$ ；删除或不再使用原有的 ADD、PUSH、POP 等指令。对于本项目而言，SUB 是实现 GCD 运算的关键指令。

3.2 指令框图及分析

本课设的指令执行过程可以分为取指译码、指令执行和返回取指三个阶段。主程序负责初始化中断系统并循环等待；中断发生后，CPU 根据中断向量进入 40H 单元的中断服务程序；服务程序完成数据输入、GCD 运算、结果输出和中断返回。整体流程如表 17 所示。

表 17 程序流程框图文字说明

流程阶段	具体内容
主程序初始化	初始化 8259，设置中断向量、屏蔽字和堆栈指针 SP
主程序等待	执行 STI 开中断后，通过 JMP 13H 循环等待
中断响应	IR0 触发后，CPU 保存返回地址，并根据中断向量进入 40H
数据输入	通过 IN R1,00H 和 IN R2,00H 读入两个操作数
比较判断	使用 R3 临时保存 R2，通过减法产生 FC/FZ 标志位
循环相减	根据 BZC 判断是否交换或继续执行相减循环
结果输出	当相减结果为 0 时，输出 R1 中的 GCD 结果
中断返回	执行 IRET，从堆栈恢复返回地址并回到主程序

本课设中新增的改造指令为 SUB RD,RS。课本原有指令系统中，0000 运算入口通常对应 ADD，而本设计为了实现更相减损法求最大公约数，需要频繁完成“大数减小数”的操作，因此将该入口对应的微程序改造为减法功能，使其实现 $RD - RS \rightarrow RD$ 。

从微操作角度看，SUB 指令的执行过程如下：先将目的寄存器 RD 的内容送入暂存器 A，再将源寄存器 RS 的内容送入暂存器 B，最后由 ALU 执行 $A - B$ 运算，并将结果写回目的寄存器 RD。该指令执行后会影响到 FC 和 FZ 标志位，后续 BZC 指令正是根据这两个标志位完成条件转移。因此，SUB 不仅承担了 GCD 运算中的核心相减功能，也为程序中的大小判断和循环控制提供了依据。对应的新增指令框图如图 3 所示。

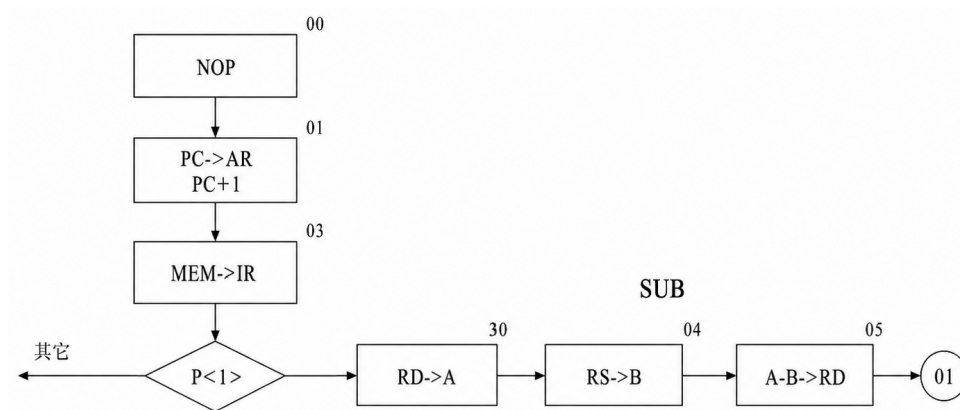


图 3 新增 SUB 指令框图

除 SUB 外，本课设还综合使用了 LDI、IN、OUT、MOV、JMP、BZC、STI、CLI 和 IRET 等指令。这些指令分别完成中断初始化、端口输入输出、寄存器传送、条件转移、开关中断和中断返回等功能。其余指令框图如图 4 所示。

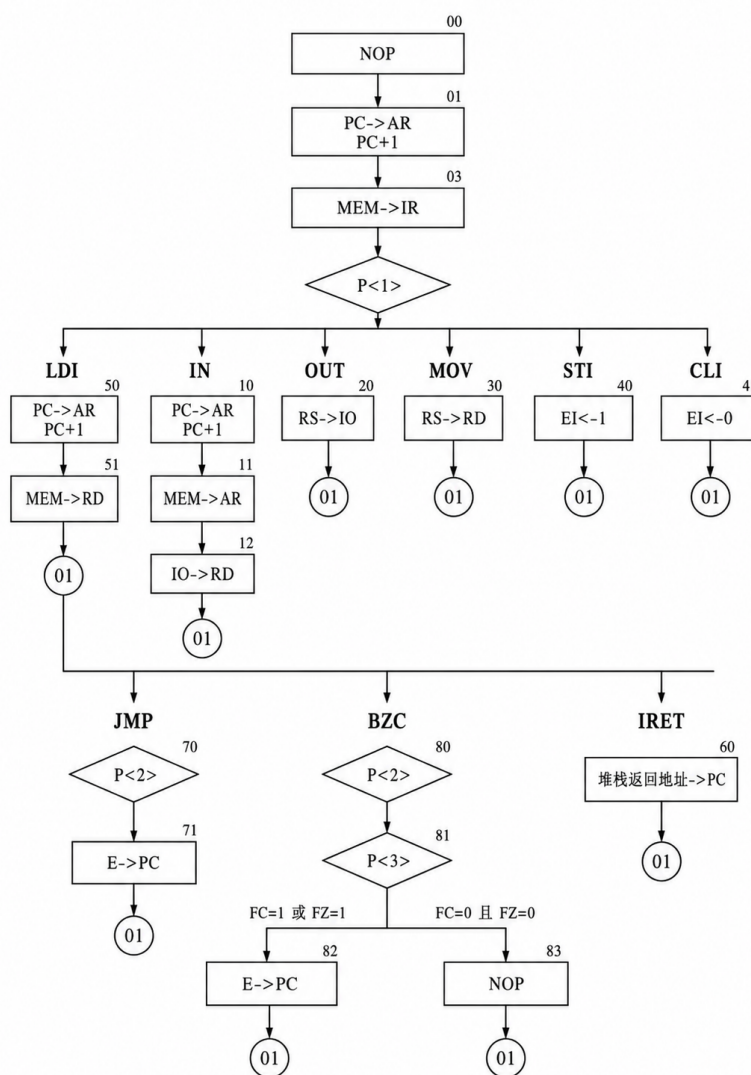


图 4 其余指令框图

从指令框图可以看出，所有机器指令都要先经过公共取指过程，再由 P<1> 完成指令译码并进入相应微程序分支。其中，BZC 需要结合 P<2> 形成有效地址，并通过 P<3> 判断 FC/FZ 标志位；STI、CLI 和 IRET 则体现了本课设中的中断控制和返回机制。

在 GCD 运算过程中，程序没有使用专门的比较指令，而是利用 SUB 产生 FC 和 FZ 标志位，再通过 BZC 完成分支控制。这样既符合复杂模型机的指令执行方式，也能较好地体现微程序控制和标志位判别之间的联系。

3.3 指令系统对应微程序二进制代码实现及分析

本课设的机器指令最终由控制存储器中的微程序完成。实验中写入的微程序以 \$M 地址数据的形式给出，其中地址表示控制存储器中的微地址，后面的 6 位十六进制数表示一条 24 位微指令。本设计在课本原有微程序的基础上进行修改，主要改造内容是加入 SUB RD,RS 指令的执行流程，使模型机能够完成更相减损法所需的减法操作。

表 18 微程序二进制代码表

地址	十六进制表示	高五位	S3~S0	A 字段	B 字段	C 字段	MA5~MA0	含义
00	0001C1	00000	0000	000	000	111	000001	CLI, 关中断
01	000102	00000	0000	000	000	100	000010	P<4> 中断判别
02	006D43	00000	0000	110	110	101	000011	PC→AR, PC+1
03	107070	00010	0000	111	000	001	110000	MEM→IR, P<1>
04	002405	00000	0000	010	010	000	000101	RS→B
05	05B201	00000	1011	011	001	000	000001	A-B→RD
06	002407	00000	0000	010	010	000	000111	RS→B
07	013201	00000	0010	011	001	000	000001	ALU→RD
08	106009	00010	0000	110	000	000	001001	MEM→AR
09	183001	00011	0000	011	000	000	000001	I/O→RD
0A	106010	00010	0000	110	000	000	010000	MEM→AR
0B	000001	00000	0000	000	000	000	000001	NOP
0C	103001	00010	0000	011	000	000	000001	MEM→RD
0D	200601	00100	0000	000	011	000	000001	RD→MEM
0E	005341	00000	0000	101	001	101	000001	ALU→PC
0F	0000CB	00000	0000	000	000	011	001011	P<3> 条件判别
10	280401	00101	0000	000	010	000	000001	RS→I/O
11	103001	00010	0000	011	000	000	000001	MEM→RD
12	200413	00100	0000	000	010	000	010011	RS→MEM

地址	十六进制表示	高五位	S3~S0	A 字段	B 字段	C 字段	MA5~MA0	含义
13	001A14	00000	0000	001	101	000	010100	SP→A
14	06C201	00000	1101	100	001	000	000001	ALU→SP
15	064216	00000	1100	100	001	000	010110	ALU→SP
16	006A17	00000	0000	110	101	000	010111	SP→AR
17	103001	00010	0000	011	000	000	000001	MEM→RD
18	064219	00000	1100	100	001	000	011001	ALU→SP
19	006A1A	00000	0000	110	101	000	011010	SP→AR
1A	105141	00010	0000	101	000	101	000001	MEM→PC
1B	005341	00000	0000	101	001	101	000001	ALU→PC
1C	10101D	00010	0000	001	000	000	011101	MEM→A
1D	10608C	00010	0000	110	000	010	001100	MEM→AR, P<2>
1E	10601F	00010	0000	110	000	000	011111	MEM→AR
1F	101020	00010	0000	001	000	000	100000	MEM→A
20	10608C	00010	0000	110	000	010	001100	MEM→AR, P<2>
22	406A23	01000	0000	110	101	000	100011	SP→AR
23	200C24	00100	0000	000	110	000	100100	PC→MEM
24	001A25	00000	0000	001	101	000	100101	SP→A
25	06C226	00000	1101	100	001	000	100110	ALU→SP
26	406027	01000	0000	110	000	000	100111	MEM→AR
27	105142	00010	0000	101	000	101	000010	MEM→PC
28	101029	00010	0000	001	000	000	101001	MEM→A
29	00282A	00000	0000	010	100	000	101010	RI→B
2A	04E22B	00000	1001	110	001	000	101011	ALU→AR
2B	04928C	00000	1001	001	001	010	001100	ALU→A, P<2>
2C	10102D	00010	0000	001	000	000	101101	MEM→A
2D	002C2E	00000	0000	010	110	000	101110	PC→B
2E	04E22F	00000	1001	110	001	000	101111	ALU→AR
2F	04928C	00000	1001	001	001	010	001100	ALU→A, P<2>
30	001604	00000	0000	001	011	000	000100	RD→A
31	001606	00000	0000	001	011	000	000110	RD→A
32	006D48	00000	0000	110	110	101	001000	PC→AR, PC+1

地址	十六进制表示	高五位	S3~S0	A 字段	B 字段	C 字段	MA5~MA0	含义
33	006D4A	00000	0000	110	110	101	001010	PC→AR, PC+1
34	003401	00000	0000	011	010	000	000001	RS→RD
35	000035	00000	0000	000	000	000	110101	NOP
36	006D51	00000	0000	110	110	101	010001	PC→AR, PC+1
37	000181	00000	0000	000	000	110	000001	STI, 开中断
38	0001C1	00000	0000	000	000	111	000001	CLI, 关中断
39	006A12	00000	0000	110	101	000	010010	SP→AR
3A	001A15	00000	0000	001	101	000	010101	SP→A
3B	001A18	00000	0000	001	101	000	011000	SP→A
3C	006D5C	00000	0000	110	110	101	011100	PC→AR, PC+1
3D	006D5E	00000	0000	110	110	101	011110	PC→AR, PC+1
3E	006D68	00000	0000	110	110	101	101000	PC→AR, PC+1
3F	006D6C	00000	0000	110	110	101	101100	PC→AR, PC+1

3.3.1 微程序分析

从表 18 可以看出, 本课设并没有重新编写整套复杂模型机微程序, 而是在课本原有微程序基础上进行局部修改。公共取指、译码、中断判别、输入输出、程序转移和中断返回等基本流程仍沿用原有设计, 保证了模型机原有指令系统的完整性。

本设计的重点修改集中在 **SUB RD,RS** 指令。其微程序执行路线为:

30H → 04H → 05H → 01H

其中, 30H 单元完成 $RD \rightarrow A$, 将目的寄存器内容送入暂存器 A; 04H 单元完成 $RS \rightarrow B$, 将源寄存器内容送入暂存器 B; 05H 单元完成 $A - B \rightarrow RD$, 由 ALU 执行减法并将结果写回目的寄存器。执行完 05H 后, 下一微地址回到 01H, 重新进入公共取指流程。

本课设最关键的微指令是 **\$M 05 05B201**。该微指令中, S3~S0 字段选择减法功能, 使 ALU 执行 $A - B$; A 字段控制结果写回寄存器; B 字段控制 ALU 输出送总线。因此, 该微指令直接决定了 SUB 是否能够正确实现 $RD - RS \rightarrow RD$ 。

SUB 执行后会影响到 FC 和 FZ 标志位。FC 反映减法是否发生借位, FZ 反映结果是否为 0。中断服务程序中的 BZC 根据 FC/FZ 判断是否跳转, 因此 SUB 不仅完成减法运算, 也为 GCD 程序中的大小比较、交换分支和循环结束提供了判断依据。

综上, 本课设对课本微程序的主要修改是: 在保留原有复杂模型机取指、译码、转移、I/O 和中断控制流程的基础上, 将运算入口改造为 **SUB RD,RS**, 使模型机能够支持更相减损法所需的减法运算。

3.4 机器程序及分析

本设计的机器程序由三部分组成：主程序、中断向量和中断服务程序。主程序从 00H 开始，主要完成 8259 初始化、SP 设置、开中断和循环等待；中断向量位于 30H 单元；中断服务程序从 40H 开始，负责输入数据、执行 GCD 运算、输出结果并中断返回。

3.4.1 汇编源代码

为了便于理解程序逻辑，先将机器程序还原为汇编形式，如下所示。

```

1  ; 主程序 00H
2  00: LDI R0,13H
3  02: OUT C0H,R0          ; 8259初始化
4  04: LDI R0,30H
5  06: OUT C1H,R0          ; 设置中断向量相关信息
6  08: LDI R0,03H
7  0A: OUT C1H,R0
8  0C: LDI R0,FEH
9  0E: OUT C1H,R0          ; 只开放IRO
10 10: LDI SP,A0H          ; 设置堆栈指针
11 12: STI                  ; 开中断
12 13: JMP 13H             ; 等待中断
13
14 ; 中断向量
15 30: 40H                 ; IRO入口地址
16
17 ; 中断服务程序 40H
18 40: CLI                  ; 关中断
19 41: IN R1,00H            ; 输入第一个数
20 43: IN R2,00H            ; 输入第二个数
21
22 45: MOV R3,R2
23 46: SUB R3,R1            ; 比较R2和R1
24 47: BZC 51H              ; R2<=R1则交换
25
26 49: SUB R2,R1            ; 大数减小数
27 4A: BZC 4EH              ; 相减为0则输出
28 4C: JMP 45H              ; 继续循环
29
30 4E: OUT 40H,R1           ; 输出GCD
31 50: IRET                 ; 中断返回
32
33 51: MOV R0,R1
34 52: MOV R1,R2
35 53: MOV R2,R0            ; 交换R1和R2
36 54: JMP 49H

```

从汇编程序可以看出，主程序不直接参与 GCD 运算，而是完成中断系统初始化后进入等待循环。真正的 GCD 计算放在 40H 单元开始的中断服务程序中。服务程序先关中断，再输入两个数，通过 SUB 和 BZC 实现比较、交换和循环相减，最后将结果从端口 40H 输出。

本程序中各寄存器的分工如下：R1 保存第一个输入数，并在最后保存 GCD 结果；R2 保存第二个输入数并参与相减；R3 用作比较临时寄存器；R0 用作交换临时寄存器。

3.4.2 程序流程图

机器程序的执行流程可分为主程序流程和中断服务程序流程。主程序负责初始化并等待外部中断；按下 KK/IR0 后，CPU 根据 30H 单元中的中断向量进入 40H 单元，开始执行中断服务程序。程序流程如图 5 所示。

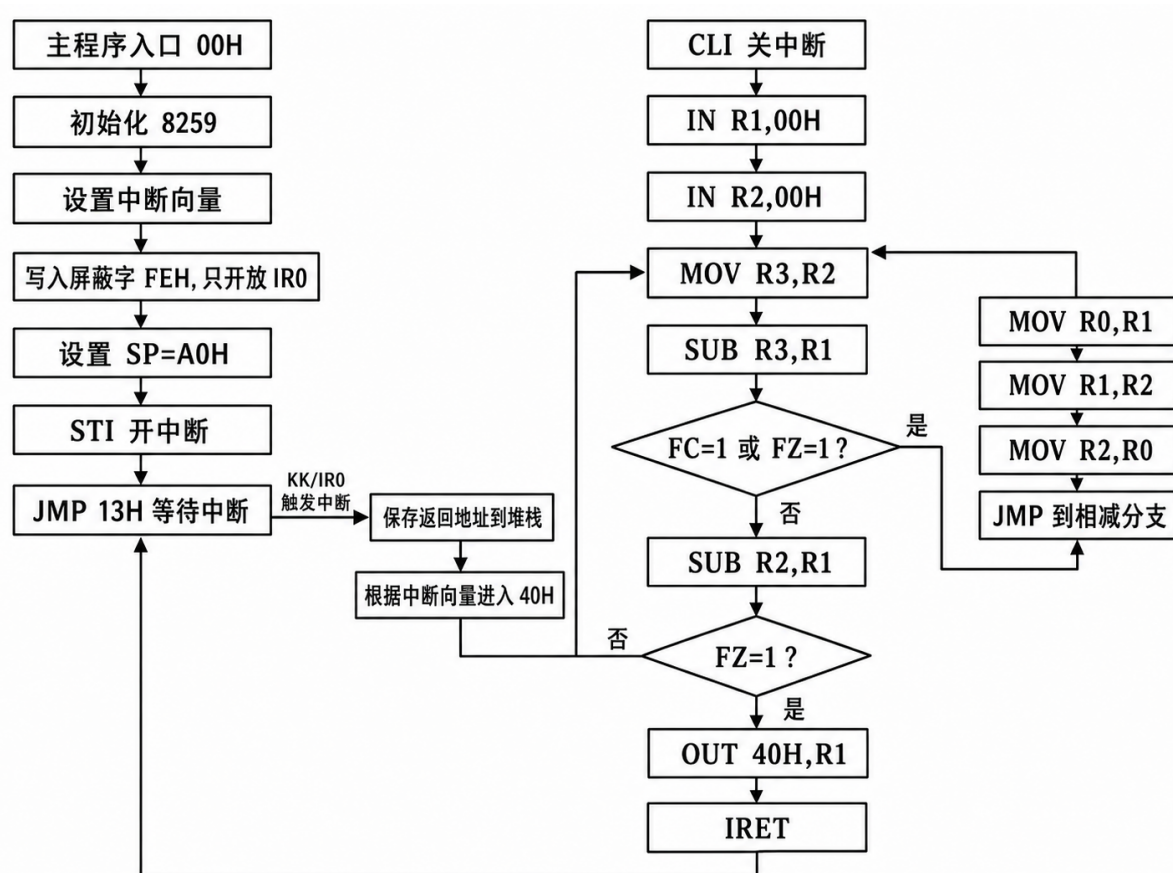


图 5 机器程序流程图

从流程上看，本程序的核心不是简单顺序执行，而是依靠中断完成主程序与服务程序之间的切换。主程序通过 STI 开中断后停留在等待循环；中断服务程序完成 GCD 运算后，通过 IRET 从堆栈恢复返回地址，使 CPU 回到主程序继续等待下一次中断。

3.4.3 机器码

本课设写入主存的机器码如下。双字节指令需要连续两个存储单元共同组成一条完整指令，例如 JMP、BZC、LDI、IN 和 OUT 等。

```
1 ; 主程序入口 00H
2 $P 00 60
3 $P 01 13 ; LDI R0,13H
4 $P 02 30
5 $P 03 C0 ; OUT C0H,R0
6
7 $P 04 60
8 $P 05 30 ; LDI R0,30H
9 $P 06 30
10 $P 07 C1 ; OUT C1H,R0
11
12 $P 08 60
13 $P 09 03 ; LDI R0,03H
14 $P 0A 30
15 $P 0B C1 ; OUT C1H,R0
16
17 $P 0C 60
18 $P 0D FE ; LDI R0,FEH
19 $P 0E 30
20 $P 0F C1 ; OUT C1H,R0
21
22 $P 10 63
23 $P 11 A0 ; LDI SP,A0H
24 $P 12 70 ; STI
25 $P 13 E0
26 $P 14 13 ; JMP 13H
27
28 ; 中断向量
29 $P 30 40 ; IRO入口为40H
30
31 ; 中断服务程序 40H
32 $P 40 80 ; CLI
33
34 $P 41 21
35 $P 42 00 ; IN R1,00H
36 $P 43 22
37 $P 44 00 ; IN R2,00H
38
39 $P 45 4B ; MOV R3,R2
40 $P 46 07 ; SUB R3,R1
41 $P 47 F0
42 $P 48 51 ; BZC 51H
43
```

```

44 $P 49 06      ; SUB R2,R1
45 $P 4A F0
46 $P 4B 4E      ; BZC 4EH
47 $P 4C E0
48 $P 4D 45      ; JMP 45H
49
50 $P 4E 34
51 $P 4F 40      ; OUT 40H,R1
52 $P 50 B0      ; IRET
53
54 $P 51 44      ; MOV R0,R1
55 $P 52 49      ; MOV R1,R2
56 $P 53 42      ; MOV R2,R0
57 $P 54 E0
58 $P 55 49      ; JMP 49H

```

其中，\$P 30 40 是本程序中最关键的地址设置之一，表示 IR0 中断发生后转入 40H 单元执行服务程序。\$P 13 E0 和 \$P 14 13 共同构成 JMP 13H，使主程序保持等待状态。\$P 47 F0、\$P 48 51 构成 BZC 51H，用于进入交换分支；\$P 4A F0、\$P 4B 4E 构成 BZC 4EH，用于判断相减结果是否为 0 并转入输出部分。

屏蔽字 FEH 的二进制形式为：

11111110B

在 8259 屏蔽字中，0 表示允许中断，1 表示屏蔽中断。因此 FEH 表示只开放 IR0，屏蔽 IR1 至 IR7，符合本课设通过 KK/IR0 触发 GCD 运算的设计要求。

四、实验步骤

4.1 微程序写入及校验

1、写入微程序

打开 TD-CMA 复杂模型机联机软件，选择“转储”——“装载”功能，将微程序文件装入实验系统。微程序采用 \$M 地址数据的格式写入控制存储器。本课设中需要重点关注 SUB RD,RS 对应的微程序，其主要路线为：

30H → 04H → 05H → 01H

其中 30H 完成 $RD \rightarrow A$ ，04H 完成 $RS \rightarrow B$ ，05H 完成 $A - B \rightarrow RD$ 。

2、校验微程序

装入完成后，通过“刷新指令区”读出控制存储器内容，与设计表逐项比对。重点检查 \$M 30 001604、\$M 04 002405 和 \$M 05 05B201 是否写入正确。其中 05H 单元决定 ALU 是否执行减法，如果该单元错误，GCD 运算和后续 FC/FZ 判断都会出错。

4.2 机器程序写入及校验

1、写入机器程序

机器程序采用 \$P 地址数据的格式写入主存。写入时按地址顺序装入主程序、中断向量和中断服务程序。

本课设机器程序地址安排如下：

地址范围	内容
00H-14H	主程序，完成 8259 初始化、设置 SP、开中断和等待
30H	中断向量，存放服务程序入口 40H
40H-55H	中断服务程序，完成输入、GCD 运算、输出和返回

写入完成后，将 PC 置为 00H，准备从主程序入口运行。

2、校验机器程序

机器程序校验时主要检查双字节指令和关键跳转地址。重点包括：\$P 13 E0、\$P 14 13 构成 JMP 13H，用于主程序等待；\$P 30 40 为中断向量；\$P 47 F0、\$P 48 51 构成 BZC 51H，用于进入交换分支；\$P 4A F0、\$P 4B 4E 构成 BZC 4EH，用于跳转到输出位置；\$P 50 B0 为 IRET，用于中断返回。

4.3 程序运行及调试

1、运行主程序

将 PC 置为 00H 后运行程序。主程序依次完成 8259 初始化、屏蔽字设置、SP 初始化和开中断操作，随后执行 JMP 13H 进入等待循环。若不能停留在等待状态，应检查 13H、14H 单元的跳转指令是否正确。

2、触发中断

主程序等待后，按下 KK/IR0 触发中断。若初始化和中断向量设置正确，CPU 应进入 40H 单元执行服务程序。服务程序先执行 CLI 关中断，再通过 IN R1,00H 和 IN R2,00H 输入两个数。

3、观察运算和返回

单步运行中断服务程序，重点观察 R1、R2、R3 以及 FC/FZ 标志位的变化。程序通过 SUB 和 BZC 完成比较、交换和循环相减。当结果产生后，执行 OUT 40H,R1 输出最大公约数，最后通过 IRET 返回主程序等待循环。

五、实验结果及分析

本课设完成微程序和机器程序写入后，对中断 GCD 程序进行了运行测试。实验时先运行主程序，使 CPU 完成 8259 初始化、SP 设置和开中断操作，并停留在 JMP 13H 等待循环中。随后通过 KK/IR0 触发中断，观察程序是否能够进入 40H 单元的中断服务程序，并完成输入、计算、输出和返回过程。

图 6 为程序运行过程中的实验截图。实验中主要观察 R1、R2、R3、ALU、PC、IR、AR 以及 OUT 端口的变化，用于判断 GCD 运算过程是否正确。

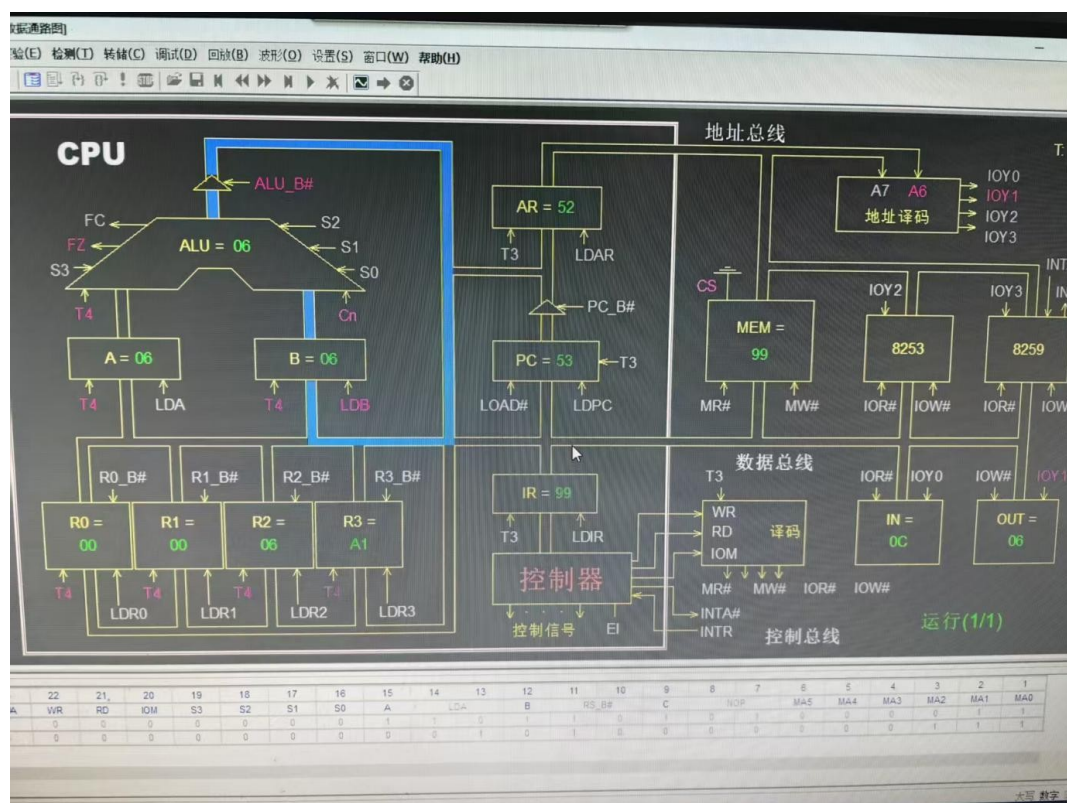


图 6 中断 GCD 程序运行结果截图

5.1 演示程序一

第一组测试数据选择 12H 和 0CH，分别对应十进制下的 18 和 12。程序运行后输出 06H，对应十进制 6。测试结果如表 19 所示。

表 19 演示程序一测试结果

输入 1	输入 2	程序输出	十进制含义	结果判断
12H	0CH	06H	18, 12, 6	正确

由数学计算可知：

$$\text{gcd}(18, 12) = 6$$

程序输出结果为 06H，与理论结果一致，说明程序能够正确完成比较、交换、循环相减和输出过程。

5.2 演示程序二

第二组测试数据选择 12H 和 02H，分别对应十进制下的 18 和 2。程序运行后输出 02H，对应十进制 2。测试结果如表 20 所示。

表 20 演示程序二测试结果

输入 1	输入 2	程序输出	十进制含义	结果判断
12H	02H	02H	18, 2, 2	正确

由数学计算可知：

$$\text{gcd}(18, 2) = 2$$

程序输出结果为 02H，与理论结果一致，说明程序在一个数能够整除另一个数的情况下，也可以正确结束循环并输出结果。

5.3 结果分析

两组实验结果汇总如表 21 所示。

表 21 实验结果汇总

测试组	输入 1	输入 2	程序输出	理论结果	结果判断
演示程序一	12H	0CH	06H	06H	正确
演示程序二	12H	02H	02H	02H	正确

从实验结果可以看出，主程序能够在等待状态下响应 IR0 中断，并正确进入 40H 单元的中断服务程序。服务程序能够通过 IN 指令读入两个数据，利用 SUB 和 BZC 完成更相减损法求最大公约数，最后通过 OUT 40H, R1 输出结果，并执行 IRET 返回主程序。两组测试结果均与理论值一致，说明本课设设计的中断 GCD 程序运行正确。

六、实验问题及思考

1. 当前所实现的计算机是否完整？如果不完整，还缺少哪些部件？

答：当前所实现的计算机还不是一台完整意义上的通用计算机。本课设主要实现了基本运算、寄存器传送、输入输出、程序转移和中断处理等功能，能够完成中断触发的 GCD 运算。但如果要进一步扩展为更完整的计算机系统，还可以补充浮点运算单元、高速缓存、更多外设接口以及更完善的异常处理机制，从而支持更复杂的数据处理和系统运行需求。

2. 当前所实现的计算机是否能够实现除法运算？如果能，可以通过哪些指令实现除法运算？

答：当前程序没有直接设计专门的除法指令，因此不能像高级语言一样直接完成除法运算。但从算法实现角度看，可以利用已有的减法、比较和转移指令设计除法过程。基本思路是不断用被除数减去除数，并用一个寄存器记录减法次数，直到被除数小于除

数或等于 0 为止。减法次数可以看作商，最后剩余的值可以看作余数。该过程主要依赖 SUB、BZC、JMP 和寄存器传送指令实现。

3. 当前所实现的计算机还能实现哪些更复杂的计算？请举例说明。

答：在现有指令系统基础上，计算机可以实现一些由基本运算和循环控制组合而成的计算。例如，通过反复加法可以实现乘法，通过反复减法可以实现除法，通过比较和条件转移可以实现排序、最大值和最小值查找等简单算法。本课设已经实现了更相减损法求最大公约数，说明只要合理组合 MOV、SUB、BZC、JMP 等指令，就可以完成一定复杂度的程序设计。另外，由于系统支持中断，还可以进一步设计由外部事件触发的计算或控制程序。

4. 当前所实现的计算机，指令系统中的双字长指令是如何实现的？

答：双字长指令通过两个连续的主存单元实现。第一字节通常存放操作码、寄存器字段或寻址方式字段，第二字节存放立即数、端口地址或转移目标地址等附加信息。CPU 执行双字长指令时，先取出第一字节并装入 IR，通过 $P<1>$ 完成操作码译码；随后根据该指令需要继续读取下一字节，得到数据字段或地址字段。这样就能在保持第一字节格式较短的情况下，完成 LDI、IN、OUT、JMP、BZC 等双字节指令的执行。

七、实验总结

7.1 验收答辩问题回答

1. 本课设实现了什么内容？

答：本课设基于 TD-CMA 复杂模型机，实现了一个中断触发的最大公约数 GCD 运算程序。主程序从 00H 单元开始，完成 8259 初始化、屏蔽字设置、SP 初始化和开中断操作，然后通过 JMP 13H 进入等待循环。

当 KK/IR0 触发中断后，CPU 根据中断向量进入 40H 单元的中断服务程序。服务程序输入两个数，通过更相减损法计算最大公约数，最后用 OUT 40H,R1 输出结果，并通过 IRET 返回主程序。因此，本课设不仅实现了 GCD 运算，也体现了中断响应、堆栈返回、I/O 操作、条件转移和微程序控制等内容。

2. 课设中中断的作用是什么？

答：中断的作用是触发 GCD 运算。主程序本身不直接计算 GCD，而是在完成初始化后等待外部中断。当按下 KK/IR0 后，CPU 暂停当前等待状态，通过中断向量进入 40H 单元执行服务程序。

这种设计体现了中断的基本思想：CPU 平时执行主程序，外部事件发生时自动转入对应服务程序，处理结束后再通过 IRET 回到原程序。相比直接用 JMP 跳转，中断方式更能体现“外部事件触发处理”的特点。

3. 主程序和子程序分别实现了什么？

答：主程序负责初始化和等待中断，主要完成 8259 初始化、设置屏蔽字 FEH、设置 SP 为 A0H、执行 STI 开中断，并通过 JMP 13H 形成等待循环。

子程序在本课设中指中断服务程序，从 40H 单元开始执行。它负责输入两个数，并使用 MOV、SUB、BZC、JMP 等指令实现更相减损法。计算结束后，程序将 R1 中的最大公约数输出到端口 40H，最后执行 IRET 返回主程序。简单来说，主程序负责“初始化和等待”，中断服务程序负责“输入、计算、输出和返回”。

4. 堆栈中存储的是什么？

答：本课设中，堆栈主要存储中断返回地址。CPU 响应中断时，需要保存被中断程序原本应该继续执行的位置；服务程序执行完 IRET 后，再从堆栈中恢复该地址到 PC，使程序回到主程序继续运行。

堆栈中不是存放 GCD 结果。GCD 结果保存在 R1 中，并通过 OUT 40H,R1 输出。本设计也没有完整保存全部通用寄存器现场，所以服务程序入口先执行 CLI 关中断，避免嵌套中断影响运算过程。

5. BZC 这条指令的结构以及对应的含义是什么？

答：BZC 是条件转移指令，格式为 1111 M ** + D。其中，高四位 1111 是操作码，M 表示寻址方式，低两位一般不用，第二字节 D 是地址字段。执行时，CPU 根据 M 和 D 形成有效地址 E，再根据标志位决定是否转移。

BZC 的含义是：当 FC=1 或 FZ=1 时，将有效地址 E 装入 PC；如果 FC 和 FZ 都不为 1，则不跳转，程序继续顺序执行。以本课设中的 \$P 47 F0、\$P 48 51 为例，F0H 表示 BZC 且 M=00，为直接寻址；第二字节 51H 是目标地址。因此这两字节组成 BZC 51H，表示条件成立时跳转到 51H 单元。

6. \$P 46 07 和 \$P 47 F0 是什么寻址方式？改成相对寻址如何改？

答：\$P 46 07 表示在 46H 单元写入机器码 07H。在本课设中，07H 对应 SUB R3,R1，属于寄存器之间的运算指令，操作数由 RS 和 RD 字段给出，可以理解为寄存器寻址。它不是带 M 字段的转移类指令，因此不涉及直接寻址、间接寻址、变址寻址或相对寻址。

\$P 47 F0 必须和下一字节 \$P 48 51 合起来看，二者共同构成 BZC 51H。F0H 的操作码为 1111，表示 BZC；M=00，表示直接寻址；第二字节 51H 是地址字段，因此条件成立时 PC 跳转到 51H。

若改为相对寻址，需要把 M 字段由 00 改为 11，所以第一字节由 F0H 改为 FCH。该指令位于 47H-48H，取完第二字节后 PC 指向 49H，目标地址仍为 51H，位移量为 08H。因此可改为 \$P 47 FC、\$P 48 08。

7. 你在原课本给出的中断程序一章进行了什么自己的修改？

答：本课设是在课本中断程序实验基础上的扩展。原实验主要演示中断响应和中断返回，本设计保留了 8259 初始化、中断向量设置、SP 设置、开中断、关中断和 IRET 返回等基本框架，但重新设计了中断服务程序的功能。

主要修改有三点。第一，将原本偏演示性的中断服务程序改为 GCD 运算程序，中断发生后不再只是简单显示或固定操作，而是输入两个数并计算最大公约数。第二，加入 IN R1,00H 和 IN R2,00H，实现从端口读入两个操作数。第三，修改了原指令系统中的运算入口，将原 0000 入口对应的微程序改为减法功能，实现 SUB RD,RS，即 $RD - RS \rightarrow RD$ 。

在本程序中，06H 和 07H 都按 SUB 使用，分别用于 SUB R2,R1 和 SUB R3,R1。

此外，程序还明确了寄存器分工：R1 保存第一个输入数和最终结果，R2 参与相减，R3 用作比较临时寄存器，R0 用作交换临时寄存器。程序结束时使用 IRET 返回，而不是普通 JMP，因此保留了真正的中断返回机制。

7.2 实验心得体会

通过本次课程设计，我对复杂模型机的运行过程有了更加完整的理解。以前学习计算机组成原理时，指令格式、寄存器、ALU、标志位、微程序、中断和堆栈这些内容比较分散，更多停留在概念层面。完成这个中断 GCD 程序后，可以明显感觉到这些知识点之间是相互联系的。机器程序能否正确执行，不仅取决于机器码写得对不对，还取决于底层微程序能否产生正确的控制信号。

在 GCD 算法实现中，我最大的体会是模型机上的程序设计和高级语言程序设计差别很大。高级语言中可以直接写 if 判断和 while 循环，但在模型机中，判断条件需要通过一次实际运算产生。比如本程序中没有直接比较两个寄存器大小的指令，只能通过 SUB 产生 FC 和 FZ 标志位，再由 BZC 根据标志位决定是否跳转。这个过程让我更清楚地理解了条件转移指令的本质。

在中断部分，我理解了主程序和中断服务程序之间的关系。主程序不是直接调用服务程序，而是在完成初始化后等待外部中断。当 KK/IR0 被按下时，CPU 响应中断，保存返回地址，并通过中断向量进入服务程序。服务程序结束后，必须使用 IRET 从堆栈恢复返回地址，而不是简单地用 JMP 跳回主程序。这个过程体现了中断机制和普通程序跳转之间的区别。

调试过程中，比较容易出错的地方主要有三类。第一类是双字节指令的第二字节容易写错，尤其是 JMP 和 BZC 的目标地址。第二类是 SUB 与课本原 ADD 入口容易混淆，如果微程序没有真正改成减法，整个 GCD 逻辑就无法成立。第三类是中断相关设置容易遗漏，例如 30H 单元中断向量、FEH 屏蔽字和 SP 初始化都很关键。通过单步运行，观察 PC、寄存器、FC、FZ 的变化，可以逐步定位这些错误。

总的来说，本次课程设计实现了从中断触发、数据输入、GCD 运算、结果输出到中断返回的完整流程。通过这个项目，我不仅复习了复杂模型机指令格式和微指令格式，也加深了对微程序控制、中断机制、堆栈返回和条件转移的理解。相比单独学习书本知识，这次实验更直观地体现了计算机硬件是如何一步一步执行程序，也让我对计算机组成原理这门课有了更具体的认识。